



TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA:	Programação Web II (2026.2)	PROFESSOR:	Lucas Santos de Oliveira
DISCENTE:	Thiago Ferreira Prates Neves	MATRÍCULA:	202511865
POLO:	UAB Brumado - BA	SEMESTRE:	4º Semestre
ATIVIDADE:	Exercícios cap. 6, 7 e 8	VENCIMENTO:	29/09/2026 - 23:59

EXERCÍCIOS - CAPÍTULOS 6, 7 E 8

Map, Set, let e const no ECMAScript 6

1. Exercício 1 - Comprinhas online

MAP

ENUNCIADO ORIGINAL - TRANSCRIÇÃO LITERAL

PDF, página 1

Implemente o método 'calculaValorTotalDaCompra' que recebe quatro parâmetros:

- produtos: Lista com o nome dos produtos comprados
- cidade: String que representa o nome da cidade
- caixa: Mapa que contém relação de produtos e preços
- fretes: Mapa que contém relação de cidades e fretes

Calcule o preço total da compra junto com o frete de acordo com as tabelas a seguir:

Produtos:

Produto	Preço
Arroz	7.10
Feijão	2.30
Macarrão	4.70
Refrigerante	3.00

Fretes:

Cidade	Frete
São Paulo	10.10
Rio de Janeiro	12.30
Brasília	14.70
Outros	13.00

- Exemplo: calculaValorTotalDaCompra(['Arroz'], 'São Paulo', caixa, fretes) → 7.20

Resposta detalhada

A estrutura **Map** é apropriada porque relaciona chaves e valores. Em **caixa**, o nome do produto funciona como chave e o preço como valor. Em **fretes**, a cidade é a chave e o valor do frete é o dado associado.

A função recebe os quatro parâmetros solicitados. Primeiro, `total` começa em zero. O laço `for...of` percorre os produtos e soma cada preço consultado com `caixa.get(produto)`. Depois, `fretes.has(cidade)` verifica se existe um frete específico para a cidade; se não existir, é utilizado o valor da chave `'Outros'`.

Logo, a regra implementada é **soma dos produtos + frete**. Para Arroz e São Paulo, os dados das tabelas fornecem **7.10 + 10.10 = 17.20**.

Observação: o PDF registra 7.20 no exemplo. A transcrição literal acima mantém esse valor exatamente como aparece no material; porém, ele é incompatível com os preços da própria tabela. A solução segue a regra escrita no exercício e retorna 17.20.

```
const caixa = new Map([
  ['Arroz', 7.10],
  ['Feijão', 2.30],
  ['Macarrão', 4.70],
  ['Refrigerante', 3.00]
]);

const fretes = new Map([
  ['São Paulo', 10.10],
  ['Rio de Janeiro', 12.30],
  ['Brasília', 14.70],
  ['Outros', 13.00]
]);

function calculaValorTotalDaCompra(produtos, cidade, caixa, fretes) {
  let total = 0;
  for (const produto of produtos) {
    total += caixa.get(produto);
  }
  if (fretes.has(cidade)) {
    total += fretes.get(cidade);
  } else {
    total += fretes.get('Outros');
  }
  return total;
}

console.log(calculaValorTotalDaCompra(['Arroz'], 'São Paulo', caixa, fretes));
// 17.20
```

2. Exercício 2 - Retirando o excesso

SET

ENUNCIADO ORIGINAL - TRANSCRIÇÃO LITERAL

PDF, página 2

Implemente o método `'removeDuplicatas'` que recebe o seguinte parâmetro:

- `numeros`: lista de números inteiros positivos

O método deve retornar a lista de número sem repetições. Utilize a estrutura de dados Set para resolver este problema.

- Exemplo: removeDuplicatas([1,1,2,2,3,3]) → [1,2,3]

Resposta detalhada

Set é uma estrutura do ECMAScript 6 que mantém apenas valores únicos. Ao criar `new Set(numeros)`, as repetições existentes na lista são descartadas automaticamente.

Depois, o conjunto é percorrido com `for...of` e cada valor é inserido em **resultado**. O retorno continua sendo uma lista/array, como pede o enunciado, mas cada número aparece apenas uma vez. A ordem da primeira ocorrência é preservada.

```
function removeDuplicatas(numeros) {
  const conjunto = new Set(numeros);
  const resultado = [];

  for (const numero of conjunto) {
    resultado.push(numero);
  }

  return resultado;
}

console.log(removeDuplicatas([1, 1, 2, 2, 3, 3]));
// [1, 2, 3]
```

3. Exercício 3 - Sei tudo sobre variáveis

VAR, LET E CLOSURES

ENUNCIADO ORIGINAL - TRANSCRIÇÃO LITERAL

PDF, página 2

Considere o trecho de código abaixo e responda as questões:

```
var arrayFuncoes = [];
for(var i = 0; i < 10; i++){
  arrayFuncoes.push(function(){
    console.log(i);
  });
}

arrayFuncoes.forEach(function(funcao){
  funcao();
});
```

a) Quais os valores são exibidos no console na execução desse código? b) Como podemos ajustar o comportamento desta função utilizando ES6?

Resposta detalhada

a) O código exibe o valor **10** dez vezes. Como **i** foi declarada com `var`, existe uma única variável compartilhada por todas as funções armazenadas em `arrayFuncoes`.

As funções são criadas no laço, mas só são executadas posteriormente no `forEach`. Quando isso ocorre, o `for` já terminou e a variável compartilhada `i` já está com o valor 10. Cada função acessa essa mesma variável por meio do fechamento léxico (closure), por isso todas imprimem 10.

b) No ECMAScript 6, basta trocar `var` por `let` no contador. Em um laço `for`, `let` cria uma associação própria de `i` para cada iteração. Assim, cada função preserva o valor correspondente à sua repetição e a saída passa a ser 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9.

```
const arrayFuncoes = [];  
for (let i = 0; i < 10; i++) {  
  arrayFuncoes.push(function() {  
    console.log(i);  
  });  
}  
  
arrayFuncoes.forEach(function(funcao) {  
  funcao();  
});  
  
// Saída: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

4. Exercício 4 - Não estou entendendo mais nada

CONST E OBJETOS

ENUNCIADO ORIGINAL - TRANSCRIÇÃO LITERAL

PDF, páginas 2-3

Uma jovem programadora pensou que havia entendido como funcionava a declaração de variáveis com `let` e `const` até ver o seguinte trecho de código:

```
const jogador = {};  
jogador.nome = 'Rodrigo';  
jogador.idade = 33;  
console.log(jogador.nome + '_' + jogador.idade); // saída: Rodrigo_33
```

Ela achou que este código não funcionaria, pois havia entendido que não era possível colocar novos valores em variáveis declaradas com `const`. Por que este código funciona sem problemas?

Resposta detalhada

`const` impede a **reatribuição da variável**, isto é, impede que `jogador` passe a apontar para outra referência. Após `const jogador = {}`, uma instrução como `jogador = { nome: 'Maria' }` não seria permitida.

Entretanto, o objeto que já está referenciado continua mutável. Ao executar `jogador.nome = ...` e `jogador.idade = ...`, o programa apenas acrescenta propriedades ao mesmo objeto; não troca a referência armazenada na constante. Por isso o código funciona e produz **Rodrigo_33**.

Se a intenção fosse impedir também a alteração das propriedades do objeto, seria necessário adotar um mecanismo adicional, como `Object.freeze()`. Isso é diferente do papel de `const`.

5. Exercício 5 - Tudo fora de escopo

CONST, LET E ESCOPO

ENUNCIADO ORIGINAL - TRANSCRIÇÃO LITERAL

PDF, página 3

Refatore o código a seguir utilizando o `const` e `let`.

```
var status = [
  { codigo: 'OK', resposta: 'Sucesso' },
  { codigo: 'FAILED', resposta: 'Erro' },
  { codigo: 'PENDING', resposta: 'Pendente' }
];
var mensagem = "";
var codigoAtual = 'OK';
for (var i = 0; i < status.length; i++) {
  if (status[i].codigo === codigoAtual) {
    mensagem = status[i].resposta;
  }
}
console.log(mensagem);
```

Resposta detalhada

A escolha entre `const` e `let` depende de haver ou não reatribuição. O array `status` não recebe outra referência depois de criado e pode usar `const`. `codigoAtual` também permanece com o mesmo valor e pode usar `const`.

A variável `mensagem` começa vazia e depois recebe uma nova string quando o código é encontrado, portanto precisa de `let`. O contador `i` muda em cada iteração e também precisa de `let`. Como consequência adicional, `i` fica limitado ao bloco do laço.

Com `codigoAtual = 'OK'`, o primeiro objeto do array corresponde à condição e `mensagem` recebe **'Sucesso'**. Portanto, essa é a saída exibida no console.

```
const status = [
  { codigo: 'OK', resposta: 'Sucesso' },
  { codigo: 'FAILED', resposta: 'Erro' },
  { codigo: 'PENDING', resposta: 'Pendente' }
];

let mensagem = "";
const codigoAtual = 'OK';

for (let i = 0; i < status.length; i++) {
  if (status[i].codigo === codigoAtual) {
    mensagem = status[i].resposta;
  }
}

console.log(mensagem);
// Sucesso
```